

AED - Atividade: Análise de complexidade (Parte 2)

Professor: Thiago Goveia	Alunos (dupla):
------------------------------------	------------------------

1. Compare as duas funções n^2 e $2^{n/4}$ para vários valores de n . Determine quando a segunda se torna maior que a primeira.
2. Qual é a configuração da entrada que representa o melhor e o pior caso, em termos de número de comparações, para a busca sequencial e para a busca binária?
3. Como um algoritmo de busca binária poderia ser utilizado dentro do algoritmo `insertionSort` (em anexo) para se reduzir o número de comparações. Seria possível aplicar a busca binária no `selectionSort`? E no `bubbleSort1`? Explique suas respostas.
4. Considere as duas implementações bubblesort (`bubbleSort1` e `bubbleSort2`). Qual delas é mais eficiente em termos de número de comparações $C(n)$ e porquê? E em relação ao número de movimentações $M(n)$?
5. O algoritmo `get_k_esimo_menor` retorna o k -ésimo menor elemento de um array desordenado.

Exemplo: se $v = [8, 9, 5, 10, 3, 1]$, ele retornará 1 se $k = 1$, 3 se $k = 2$ e 5 se $k = 3$.

- a. Qual algoritmo simples de ordenação (modificado) está sendo utilizado?
 - b. Qual é a função de custo da implementação no melhor e pior caso?
 - c. Qual é a sua complexidade assintótica na notação $O(n)$ para o pior caso;
6. Determine, para os algoritmos típicos usados para fazer contas à mão, a complexidade assintótica para:
 - a. Somar 2 inteiros de N dígitos;
 - b. Multiplicar 2 inteiros de N dígitos;
 7. Os trechos de código a seguir executam operações em uma matriz $n \times n$: multiplicação, transposição e soma. Identifique cada um e apresente a sua função de custo e o comportamento assintótico na notação $O(n)$.

```
a. for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        a[i][j] = b[i][j] + c[i][j];

b. for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        for (k = a[i][j] = 0; k < n; k++)
            a[i][j] += b[i][k] * c[k][j];

c. for (i = 0; i < n - 1; i++)
    for (j = i+1; j < n; j++) {
        tmp = a[i][j];
        a[i][j] = a[j][i];
        a[j][i] = tmp;
    }
```

8. Marque verdadeiro ou falso:

- a) () A medida do tempo de execução não é uma métrica confiável, visto que pode variar de acordo com a configuração da máquina em que o algoritmo é executado
- b) () No limite assintótico Θ são avaliadas 2 constantes (c e m) para limitar a função analisada. Na notação $O(n)$ são avaliadas 3 constantes (c1, c2 e m);
- c) () A busca binária possui custo de tempo de execução $O(\log n)$, mas não pode ser aplicada a qualquer array;
- d) () O custo de uma função pode ser dado em termos do tempo de execução (quantidade de operações) ou do espaço de armazenamento utilizado;
- e) () O mais usual na análise de algoritmos é avaliar o melhor caso. O caso médio raramente é utilizado pois depende de uma análise probabilística, o que é algo complexo de se fazer;
- f) () A expressão “pior caso” se refere ao maior tamanho N de uma entrada, que pode ser executado por um algoritmo;
- g) () A notação Ômega (Ω) define o limite assintótico inferior de uma função (ou algoritmo);
- h) () Um algoritmo exponencial é sempre pior que um algoritmo polinomial.
- i) () As terminologias “função de complexidade” e “comportamento assintótico” se referem à mesma coisa: o custo de executar um algoritmo com uma dada entrada de tamanho n

9. Relacione as colunas:

_____ $O(1)$ - Constante	A. Pesquisa binária em uma lista ordenada
_____ $O(\log n)$ - Logarítmica	B. Pesquisa linear em uma lista
_____ $O(n)$ - Linear	C. Multiplicação de duas matrizes (algoritmo clássico)
_____ $O(n \log n)$ - $n \log n$	D. Ordenação por seleção (Selection Sort)
_____ $O(n^2)$ - Quadrática	E. Algoritmo para resolver problemas como torre de hanoi, 8 rainhas e caixeiro viajante.
_____ $O(n^3)$ - Cúbica	F. Ordenação por mergesort ou heapsort
_____ $O(2^n)$ ou $O(n!)$ - Exponencial	G. Acesso direto a um elemento em uma lista (por índice)

10. Obtenha as funções de custo $C(n)$ e $M(n)$ para o pior caso dos algoritmos `selectionSort` e `insertionSort`.

Dica: Primeiro, identifique para cada algoritmo, qual é o pior caso para $C(n)$ e $M(n)$.

Algoritmos:

```
public static void insertionSort(int[] v){
    int n = v.length;
    for (int i = 1; i < n; i++) {
        for(int j = i; j > 0; j--){
            if(v[j] < v[j-1]){
                troca(v, j, j-1);
            }else{
                break;
            }
        }
    }
}
```

```

public static void bubbleSort1(int[] v){
    int n = v.length;
    for (int i = 0; i < n-1; i++)
        for (int j = n-1; j > i; --j)
            if (v[j] < v[j-1])
                troca(v, j, j-1);
}

public static void bubbleSort2(int[] v){
    int n = v.length;
    for (int i = 0; i < n-1; i++)
        for (int j = n-1; j > 0; --j)
            if (v[j] < v[j-1])
                troca(v, j, j-1);
}

public static void selectionSort ( int[] v ) {
    int n = v.length;
    for( int i = 1; i <= n - 1; i ++ ) {
        int min = i ;
        for ( int j = i + 1; j <= n; j++)
            if (v[j] < v[min])
                min = j ;
        troca(v, min, i);
    }
}

public static int get_k_esimo_menor(int v[], int k){
    int i, j, mini, tmp;
    int n = v.length;
    for (i = 0; i < k; i++) {
        mini = i;
        for (j = i+1; j < n; j++)
            if (v[j]<v[mini])
                mini = j;
        tmp = v[i];
        v[i] = v[mini];
        v[mini] = tmp;
    }
    return v[k-1];
}

```